

Big Data Pipeline Monitor

Architektúra distribovanej aplikácie *qr-blocky*

Jozef Waldhauser

MSWA

2026 léto

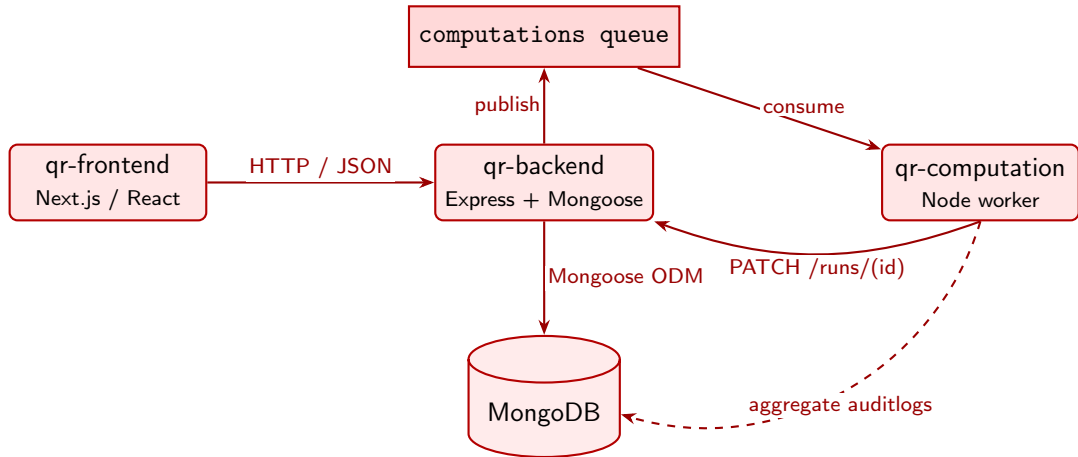
- 1 Pôvodná idea
- 2 Architektúra
- 3 Thought process
- 4 Backend
- 5 Doménový model
- 6 Vstupné a runtime modely
- 7 Computation module
- 8 Frontend
- 9 Zhrnutie

Prečo práve qr-blocky?

- Názov vznikol z QR kódu a pokladničného bloku.
- Frontend, **ktorý skenuje QR z bločku**.
- Backend má qr-app endpoint, každé volanie sa zaznamenáva do **auditlogov** (request log).
- Cieľ monitoringu: **agregovať počet volaní** na endpoint skenovania bločku aj na všetky ostatné qr-app endpointy.
- Nad auditlogmi beží **pipeline**, ktorá v zadanom časovom okne zagreguje počet volaní a **zapíše štatistiky späť do DB**.

Z toho vznikol monitoring framework

Aplikácia dnes eviduje **datasets** (zdroj dát), **pipelines** (čo sa agreguje a kedy), **behy** (JobRun) a **alerty** - kostra je všeobecná, ale doménovo cieľi na *auditlogs* qr-blocky služieb.



- **Synchrónna cesta:** prehliadač → API (CRUD nad datasetmi, pipelineami, stav behu).
- **Asynchrónna cesta:** API → fronta → worker → API (PATCH stavu).

Cesta k Big Data architektúre

- 1 **Frontend volá backend** - naskenovanie QR → HTTP požiadavka, jednoduchá synchrónna cesta.
- 2 **Backend musí tvoriť objekty** - Dataset, Pipeline, JobRun, Alert; klasický REST + ODM.
- 3 Pri raste objemu volaní synchrónne spracovanie nestačí ⇒ posun k **Big Data architektúre**: oddelenie zápisu, fronty a výpočtu.
- 4 **RabbitMQ ako buffer** - horizontálne škálovateľný, drží **veľký počet správ**.
- 5 **Computation module subscribuje** správy a spracúva ich **podľa konfigurácie**.
- 6 Pri extrémnych objemoch: **instance per queue** - vlastná queue a vlastná skupina workerov pre každý RabbitMQ channel.
- 7 Design architektúry pre **KEDA job** - dynamické škálovanie podľa dĺžky fronty, napr. v **Azure** (AKS + KEDA).

- Express 5, modulárne routy v `app/`: `datasets`, `pipelines`, `runs`, `alert-rules`, `alerts`.
- Perzistencia cez **Mongoose** (MongoDB) - jeden model na entitu (`Dataset`, `Pipeline`, `JobRun`, `AlertRule`, `Alert`).
- **Validácia** vstupu **Joi** schémami v každom module (`validation.js`).
- **Error handling**: per-route `try/catch`, centrálny 404 a 500 handler v `server.js`.
- Publikácia úloh do RabbitMQ cez `app/rabbitmq.js` (front computations, durable).

Oddelenie zodpovedností

Router robí validáciu a HTTP, model drží schému, business pravidlá sú v handleri (napr. kontrola `active`, stavový prechod).

Doménový model (Mongoose)

- **Dataset** - metadáta (name, owner, type: aggregation|file, aggregation.timeFrom/To).
- **Pipeline** - odkaz na dataset (datasetOid), schedule, active, pipelineVersion, lastStatus, lastRunTime.
- **JobRun** - konkrétny beh: pipelineOid, pipelineVersion, status, časy a processedRecords.
- **AlertRule** - definícia (pipelineOid, reportWhenState).
- **Alert** - vyvolaný záznam (väzba na pipeline + run, acknowledgedAt).

Pravidlo integrity

Pipeline nemôže vzniknúť nad neexistujúcim datasetom - kontroluje sa pred uložením v handleri.

Vstup pipeline a runtime stav workera

AuditLog - kolekcia auditlogs (qr-backend)

- Polia: action, endpoint, requestBody, requestQuery, ip, status, timestamp.
- Zapisuje sa **middlewareom** v app/qr-app pri každom volaní na /qr-app/* (getReceipt, saveReceipt, shareReceipt).
- **Vstup pipeline** - nad touto kolekciou beží agregácia v computation module (\$match podľa timestamp, \$group podľa action).

ComputationJob - kolekcia computationmodule (qr-computation-module)

- Polia: runId, frequency, timeInterval, cmds, receivedAt, receiveCount.
- Pri každej správe **upsert podľa** runId a \$inc receiveCount - worker vie rozlíšiť prvý príchod od duplicity.
- receiveCount > 1 $\Rightarrow$ **force re-run** (zruší naplánovaný cron a spustí calcStats okamžite).

- Samostatný Node proces s vlastným Express serverom a RabbitMQ consumerom (app/rabbit-consumer).
- Pre každú správu calcStats:
 - 1 PATCH /runs/:id → running (s startTime),
 - 2 agregácia nad MongoDB kolekciou auditlogs v zadanom časovom intervale,
 - 3 uloženie výsledku do computationstats,
 - 4 PATCH /runs/:id → successful / error.
- **Idempotencia** - upsert ComputationJob podľa runId + receiveCount.
- **Škálovateľnosť** - viac inštancií modulu paralelne konzumuje z tej istej fronty; pri raste objemu možné nasadenie **instance per channel**.
- **Odolnosť** - reconnect s exponenciálnym backoffom, prefetch(1), ack/nack.

- **Next.js** App Router - React Server Components načítavajú dáta priamo z API počas renderu (`src/lib/api.ts`).
- Stránky: `/` (dashboard), `/datasets`, `/pipelines`, `/pipelines/[id]`, `/runs`, `/runs/[id]`, `/alerts`, `/alert-rules`.
- Klientske komponenty pre akcie a živý stav: `RunPipelineButton`, `RunStatusLive`, `PipelineActiveToggle`, `AcknowledgeAlertButton`.
- **Stavy UI**: *loading* (RSC streaming), *empty* (prázdne zoznamy), *error* (chybové hlášky z `ApiRequestError`).
- Cez `next.config` rewrite na `/blocky-api-same-origin` volania z prehliadača (žiadne CORS).

Zhrnutie architektúry

- **Server-side stavový model** - endpoint `PATCH /runs/:id` povoľuje iba validné prechody stavov; neplatná zmena vracia 409. Pipeline sa spúšťa len pri `active = true`.
- **Oddelenie na tri služby** - frontend, API a computation modul fungujú nezávisle, takže výpočtové úlohy neblokujú HTTP vrstvu.
- **RabbitMQ s durable queue** - kombinácia `prefetch(1)` a `ack/nack` zabezpečuje spoľahlivé spracovanie úloh a prirodzený load balancing medzi workermi.
- **MongoDB + Mongoose + Joi** - dokumentový model dát dopĺňa dvojúrovňová validácia: Joi pre HTTP vstup a Mongoose pri ukladaní do databázy.
- **Next.js React Server Components** - server renderuje dátové výpisy priamo z API, klientské komponenty riešia len interakcie a polling stavu.

Výsledok

Návrh predstavuje jednoduchý, ale škálovateľný monitoring framework pripravený na distribuované spracovanie veľkého objemu auditlogov.

Ďakujem za pozornosť.

Otázky?